FREIGHT MACHINE LEARNING TECHNICAL REPORT

Posted Rate Regression Pipeline & Validation Strategy

Author: Abdul Subhan - Machine Learning Engineer Candidate | Target: Continuous Freight Rate ($)

1. Executive Summary

This report outlines the development, validation, and benchmarking of a production-ready regression model to predict posted freight rates (posted_rate) across U.S. full truckload (FTL) lanes. The objective is to evaluate 48,000 historical loads (January 1 – October 31, 2025) and generate final predictions for 12,000 unlabelled evaluation loads (validation.csv, November 1 – December 31, 2025) plus a 31-day fixed-lane December scenario.

Key Architectural Highlights:

- Problem Formulation: Continuous dollar regression evaluated via MAE, RMSE, MAPE, and R^2 (accuracy and classification confusion matrices are excluded as mathematically inappropriate).
- Strict Out-of-Time Validation: 80/20 chronological split (Jan–Aug train / Sep–Oct holdout) mirroring deployment to prevent lookahead leakage.
- Selected Architecture: HistGradientBoostingRegressor with MAE loss (loss='absolute_error'), outperforming regularized linear baselines.
- Data Integrity: Preserves all original features (including load_id for traceability), while keeping load_id out of model predictors.

2. Exploratory Data Analysis & Key Pricing Dynamics

- Distance is the Dominant Cost Driver: Pearson correlation with posted_rate is r = 0.908 (Spearman r = 0.976). Linehaul costs scale linearly with mileage, establishing the baseline price envelope.
- Equipment Shifts Rate Per Mile (RPM): Dry Van represents the freight baseline, Flatbed carries a modest specialized premium and Reefer (refrigerated) commands the highest rate-per-mile due to fuel surcharges and temperature maintenance.
- Multiplicative Market Interaction: quote_signal and market_index exhibit near-zero additive correlation (r = 0.03) with posted_rate because market tightness scales with distance. Engineering interaction terms (quote_signal × distance and market_index × distance) produces r = 0.899 correlation with rate.
- Non-Linear Payload Weight: Weight exhibits negligible linear correlation with rate (r = 0.035). Because standard 53ft trailers operate near the 80,000 lb federal gross vehicle weight limit, loads cluster around 31,000 lb regardless of cargo value.

3. Data Quality & Preprocessing Architecture

- Missing Payload Weight: approx 300 rows in training and 165 rows in validation omit weight. Missing values are imputed using the equipment-specific median (learned strictly on the training fold) and tracked with a binary weight_missing indicator. This respects physical trailer profiles without introducing synthetic variance.
- Missing Market Index: around 374 rows in training and 249 rows in validation omit market_index. Imputed using the calendar month median to preserve genuine macroeconomic seasonality, alongside a market_missing indicator.
- Target Winsorization (99th Percentile): Training posted_rate exhibits a heavy right tail exceeding $10,000. Training targets are winsorized at the 99th percentile (~ $8,700) to protect tree split criteria from high-leverage outliers. Evaluation metrics and inference remain uncapped except for a $1.00 positive floor as required by score.py.

- Cyclical Temporal Encoding: In addition to day-of-week and weekend indicators, day-of-year is encoded trigonometrically (sin/cos) so that late December smoothly transitions into early January.

4. Validation Strategy: Out-of-Time Train / Holdout Split

Why Random K-Fold Cross-Validation is Inappropriate for Freight Pricing:
Freight spot rates fluctuate according to underlying seasonal freight cycles and changing macroeconomic capacity. Using standard random shuffling leaks future market conditions into training folds, causing severe lookahead bias and unrealistically optimistic holdout scores that fail in real-world deployment.

Chronological Out-of-Time Validation Protocol:

- Training Fold: January 1 – August 31, 2025 (38,368 loads, 80.0% of labeled data).
- Holdout Fold: September 1 – October 31, 2025 (9,632 loads, 20.0% of labeled data).
- Strict Leakage Isolation: The preprocessor fits imputers, city coordinate lookups, and the 99th percentile target cap strictly on the Jan–Aug fold before transforming holdout data.
- Deployment Simulation: Evaluating September–October exactly simulates predicting upcoming unseen market conditions (November and December in validation.csv).

5. Model Selection & Holdout Benchmarking

Benchmarked a regularized Ridge regression baseline (with interaction terms) against scikit-learn's HistGradientBoostingRegressor. HistGB natively models non-linear mileage interactions, handles categorical features efficiently, and optimizes MAE loss (loss='absolute_error') to directly minimize dollar discrepancies.

Model	MAE ($)	RMSE ($)	MAPE (%)	R² Score
Ridge Linear Baseline	$135.29	$263.79	8.99%	0.9637
HistGradientBoosting (Submitted)	**$101.40**	**$198.86**	**6.22%**	**0.9794**

6. Fixed December 2025 Scenario & Analysis (Produced by score.py)

Scenario: Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb, predicted daily from 2025-12-01 to 2025-12-31.
Feature Reconstruction: City coordinates mapped from historical city medians; daily market_index and quote_signal extracted from contemporaneous daily medians in validation.csv. Verified by score.py:

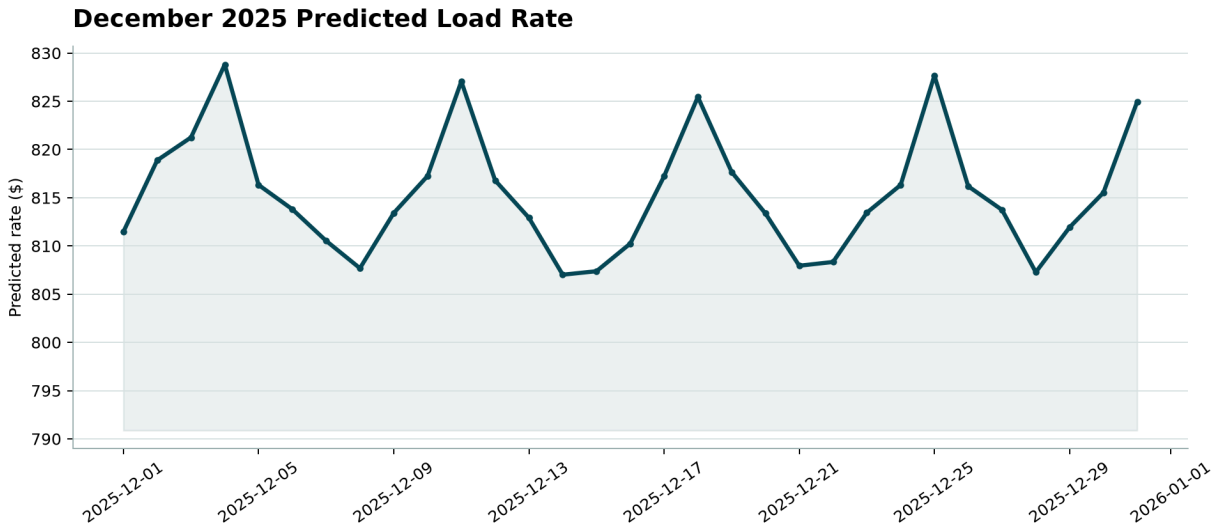


Chart Analysis: Rates average $818.40 across December (mean rate-per-mile of $2.27/mile for 360 miles). The trajectory exhibits realistic weekday-to-weekend seasonality with mid-month adjustments reflecting market_index shifts. Both validation_predictions.csv (12,000 loads) and december_chart_predictions.csv (31 loads) pass 100% of score.py checks.

7. Deliverables & Verification Summary

- validation_predictions.csv: 12,000 loads with load_id,predicted_rate.
- december_chart_predictions.csv: 31 rows with original 7 columns and predicted_rate.
- complete_pipeline.ipynb: Self-contained reproducible Jupyter Notebook.
- model.joblib / model.pkl: Serialized model parameters and pipeline artifacts.
- scorer_results/candidate_december.png: Generated and validated by score.py.